CPSC-406 Report

Angelina House
Chapman University

February 23, 2025

Abstract

Contents

1 Introduction

2 Week by Week

2.1 Week 1

Notes

Week One Notes.

Homework

Question 1: Characterize all the accepted words (i.e., describe exactly those words that get recognized).

We can first start characterizing the accepted words by looking at the given graph and gaining an understanding of the automaton. In this automaton there we have a coin machine which needs 25 cent and can only take 5 and 10 cent coins. We can see that the machine will accept the word if the sum of the coins is 25 cents. With this we can characterize the accepted words as follows:

5 5 5 5 5
5 5 5 10
5 5 10 5
5 10 5 5
10 5 5 5
10 10 5
10 5 10
5 10 10

Question 2: Characterize all the accepted words. Can you describe them via a regular expression?

We can first start charaterizing the accepted words by understanding the graph, states, and transitions. In this graph, pay moves the turnstile from locked to unlocked, push turns the turnstile from unlocked to

locked, if the turnstile is already unlocked a pay occurs then it remains unlocked and if the turnstile is locked and a push is used then nothing happens. With this understanding we can formulate that atleast one pay is needed to unlock the turnstile, any number of pay's can happen while the turnstile is unlocked and a push after a pay locked the turnstile making it need another pay to unlock again. With this we can characterize the accepted words as follows:

```
pay
pay pay
pay pay pay
pay push pay
pay pay push pay
pay pay pay push pay
```

Overall, the pattern follows:

```
pay (push pay)*
```

Reading Summary

comments and questions

2.2 Week 2

Notes

This week we covered Derterministic Finite Automata (DFA) and continued to build on our definition on automata from week one.

Homework

Question 1: Determine for the following words, if they are contained in L1, L2, or L3

The equations are:

$$L_1 := \{x01y \mid x, y \in \Sigma^*\}$$
$$L_2 := \{w \mid |w| = 2^n \text{ for some } n \in \mathbb{N}\}$$
$$L_3 := \{w \mid |w|_0 = |w|_1\}$$

Using the given chart and the given equations we can come to the following conclusions.

Word	L_1 (Contains "01")	L_2 (Length 2^n)	L_3 (Equal 0s and 1s)
$w_1 = 10011$	✓	x	x
$w_2 = 100$	x	x	x
$w_3 = 10100100$	✓	✓	×
$w_4 = 1010011100$	✓	×	✓
$w_5 = 11110000$	×	✓	✓

By following the the language equations, conditions, and comparing using the word we can determine whether the word would be accepted in the equation.

Question 2: Let Σ be some fixed alphabet and $\mathcal{A} = (Q, \Sigma, \delta, q_0, F)$ be a finite automaton. The DFA $\mathcal{A}$ can "process" a word by tracing through it via a sequence of transitions corresponding to the letters of the word. Consider the paths corresponding to the words:

$$w_1 = 0010, \quad w_2 = 1101, \quad \text{and} \quad w_3 = 1100.$$

For which of these words does their run end in the accepting state?

w_1 and w_2 end in the accepting state. Following the given graph and going through the word we can determine that w_1 and

Reading Summary and Questions

Section 2.1 introduces finite automata in a straightforward easy to comprehend way by explaining how they work and why they're useful. A finite automaton is a simple computational model that processes input by transitioning between a set number of states. It follows predefined rules to determine what state to move to based on the input it receives. The section gives real world examples where finite automata are applied, such as text searching, lexical analysis in programming languages, digital circuit design, and verifying network protocols. These machines help computers recognize patterns and process sequences efficiently.The section discusses two main types, Deterministic Finite Automata (DFA), where each state has only one possible transition per input, and Nondeterministic Finite Automata (NFA), which allows multiple possible transitions. Both are equally as powerful but NFA's are typically more flexible. This section doesn't get too technical yet it just lays the foundation for the more detailed discussions coming later. The key takeaway is that while finite automata are simple, they play an essential role in many areas of computer science, especially in recognizing and processing data.

My questions this week was regarding the complexities of finite automata. It was "Is there a point where an automata gets too complicated to the point where its not the best option anymore? If so, is there any alternatives to the automaton that might work better in this situation?"

2.3 Week 3

Notes

Homework

Homework two include two lab questions from our week two lab. The labs focus on developing and altering a DFA from code as well as understanding the varuous actions that can be done to them. In the first question, we are asked to implement the run function as well as define the A_1 and A_2 from the imagaes given. In the following question, we are asked to consider a new automaton A_3 and implement a new refuse function. This is all done within three different python files, dfa.py, dfa$_e$x01.*pyanddfa*$_e$x03.*py*.

dfa.py:

```
# dfa.py
class DFA:
    def __init__(self, Q, Sigma, delta, q0, F):
        self.Q = Q # Set of states
        self.Sigma = Sigma # Alphabet
        self.delta = delta # Transition function
        self.q0 = q0 # Initial state
```

```
        self.F = F # Accepting states

    def __repr__(self):
        return f"DFA({self.Q},\n\t{self.Sigma},\n\t{self.delta},\n\t{self.q0},\n\t{self.F})"

    def run(self, w):
        current_state = self.q0
        for symbol in w:
            if symbol not in self.Sigma:
                return False
            current_state = self.delta.get((current_state, symbol))
            if current_state is None:
                return False
        return current_state in self.F # Accept if in accepting state

    def refuse(self):
        # new dfa that refuses input
        return DFA(self.Q, self.Sigma, self.delta, self.q0, set())

# Example DFA
if __name__ == "__main__":
    Q = {'q0', 'q1'}
    Sigma = {'0', '1'}
    delta = {
        ('q0', '0'): 'q0',
        ('q0', '1'): 'q1',
        ('q1', '0'): 'q1',
        ('q1', '1'): 'q0'
    }
    q0 = 'q0' # Initial state
    F = {'q1'} # Accepting state

    dfa = DFA(Q, Sigma, delta, q0, F)
    print(dfa.run('101')) # false
    print(dfa.run('100')) # true
```

***** dfa_ex01.py: *****

```
from dfa import DFA

def generate_words():
    words = [""]
    alphabet = ['a', 'b']

    # Generate words
    for first in alphabet:
        words.append(first)
        for second in alphabet:
            words.append(first + second)
            for third in alphabet:
                words.append(first + second + third)

    return words

def __main__():
    # Define DFA A1
```

```
    A1 = DFA(
        Q={1, 2, 3, 4},
        Sigma={'a', 'b'},
        delta={
            (1, 'a'): 2, (1, 'b'): 4,
            (2, 'a'): 2, (2, 'b'): 3,
            (3, 'a'): 3, (3, 'b'): 3,
            (4, 'a'): 4, (4, 'b'): 4
        },
        q0=1,
        F={3}
    )

    # Define DFA A2
    A2 = DFA(
        Q={1, 2, 3},
        Sigma={'a', 'b'},
        delta={
            (1, 'a'): 2, (1, 'b'): 1,
            (2, 'a'): 3, (2, 'b'): 1,
            (3, 'a'): 3, (3, 'b'): 3
        },
        q0=1,
        F={3}
    )

    words = generate_words()
    automata = [A1, A2]

    # Test words on automata
    for X in automata:
        # print(f"{X.__repr__()}") # prints DFA representation
        for w in words:
            print(f"{w}: {X.run(w)}") # prints true or false
        print("\n")

__main__()
```

***** dfa_ex03.py: *****

```
from dfa import DFA

def generate_words():
    words = [""]
    alphabet = ['a', 'b']

    for first in alphabet:
        words.append(first)
        for second in alphabet:
            words.append(first + second)
            for third in alphabet:
                words.append(first + second + third)

    return words

def __main__():
```

```
    # Define DFA A
    A = DFA(
        Q={1, 2, 3, 4},
        Sigma={'a', 'b'},
        delta={
            (1, 'a'): 2, (1, 'b'): 4,
            (2, 'a'): 3, (2, 'b'): 4,
            (3, 'a'): 3, (3, 'b'): 3,
            (4, 'a'): 2, (4, 'b'): 4
        },
        q0=1, # Initial state
        F={3, 4} # Accepting states
    )

    # Define DFA A0 (refuse)
    A0 = A.refuse()

    words = generate_words()
    automata = [A, A0]

    # Test words on automata
    for X in automata:
        # print(f"{X.__repr__()}") # Prints DFA representation
        for w in words:
            print(f"{w}: {X.run(w)}") # Prints True or False
        print("\n")

__main__()
```

Reading Summary and Questions

Section 2.2 focuses on Deterministic Finite Automata (DFA), which are computational models used to process input strings and determine whether they belong to a specific language. A DFA consists of a finite set of states Q, a defined input alphabet Σ, transition rules δ that dictate how the automaton moves between states, a starting state q_0, and one or more accepting states F that signify valid inputs. The operation of a DFA is systematic: it begins in the initial state and processes an input string one symbol at a time, following a unique transition for each symbol. Once the entire input has been read, the DFA determines acceptance based on its final state. If it ends in an accepting state, the input is considered valid; otherwise, it is rejected. Formally, a DFA is defined as a five-tuple $(Q, \Sigma, \delta, q_0, F)$, where Q is the set of states, Σ is the input alphabet, δ is the transition function that maps states and inputs to new states, q_0 is the initial state, and F is the set of accepting states. DFAs are widely used in applications such as text searching, lexical analysis in programming, and pattern recognition. Their deterministic nature makes sure that for any given input, the transition path is unique making them both predictable and efficient for structured data processing.

This week I was curious about the limits of a DFA. My question was "The book emphasized that DFA's can only process one string at a time but would we ever get to the point where it would be able to process multiple strings at one time? Would it make the DFA simpler or more complex and at what point would we have to get quantum computing involved?"

3 Synthesis

4 Evidence of Participation

5 Conclusion

References

[BLA] Author, Title, Publisher, Year.